

# Introdução aos tópicos de base e terminologias essenciais segundo Engenharia de Software: Uma Abordagem Profissional

PROCESSO DE SOFTWARE — Professor: Me. Sergio Souza Novak —  
Faculdade de Engenharia de Software



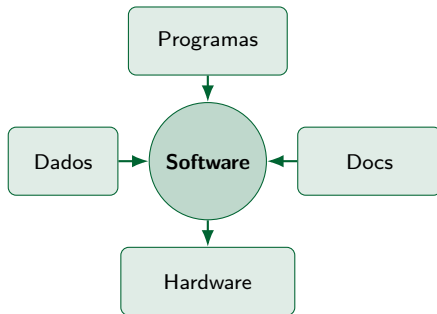
- 1 Disponibilidade de serviços e desafios
- 2 1.1 A Natureza do Software
- 3 1.2 Definição da Disciplina
- 4 1.3 O Processo de Software
- 5 Atividades de Apoio
- 6 1.3.3 Adaptação do Processo
- 7 1.4 A Prática da Engenharia de Software
- 8 1.4.1 A Essência da Prática

# 1.1 A Natureza do Software

## Conceito

Software é o conjunto de programas, dados e documentação que permite ao computador executar tarefas e resolver problemas.

- Não é um componente físico.
- Controla o funcionamento do hardware.
- Processa, armazena e transmite informações.



## Importância

Vivemos na era da informação, e o software é o principal meio de criação, processamento e distribuição dessa informação.

# O Duplo Papel do Software

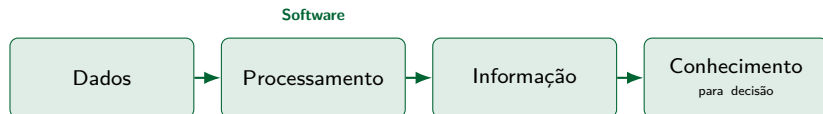


## Exemplos — Produto

- Microsoft Word, WhatsApp
- Photoshop, Sistema Bancário

## Exemplos — Veículo

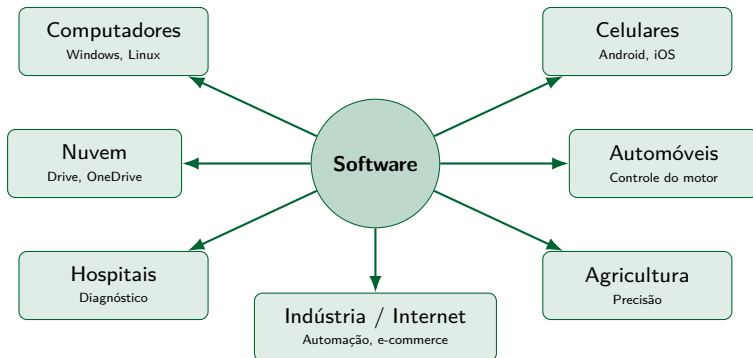
- Windows, Linux, Android
- Compiladores



O software pode:

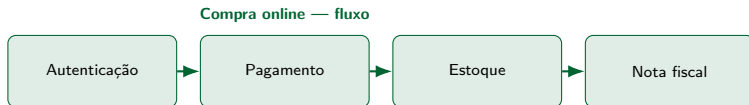
- Produzir informações;
- Armazenar dados;
- Modificar conteúdos;
- Exibir resultados;
- Compartilhar informações pela Internet.

# Onde o Software Está Presente?



# O Software Distribui Informação

O principal produto distribuído pelo software é a informação.



- Dados financeiros e comerciais
- Sistemas governamentais
- Comunicação pela Internet
- Redes sociais
- Inteligência Artificial
- Serviços em nuvem



## Equilíbrio

O mesmo poder que gera benefícios amplia a superfície de risco: segurança e ética tornam-se requisitos de projeto.

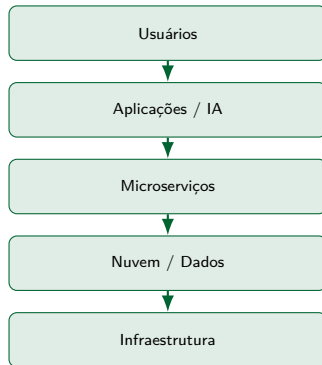


A evolução do hardware permitiu o desenvolvimento de softwares cada vez mais complexos.

# O Crescimento da Complexidade

Os sistemas atuais possuem:

- Milhões de linhas de código;
- Milhares de usuários simultâneos;
- Comunicação entre dezenas de serviços;
- Armazenamento em nuvem;
- Inteligência Artificial integrada;
- Alta disponibilidade e segurança.



## Consequência

Quanto maior a complexidade, maior a necessidade de planejamento, projeto, testes e manutenção.



Mesmo com tecnologias modernas, várias perguntas continuam atuais:

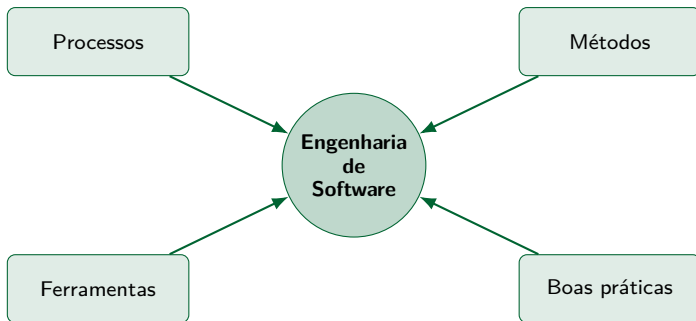
1. Por que demora tanto para ficar pronto?

2. Por que o desenvolvimento custa caro?

3. Por que ainda existem tantos erros?

4. Por que a manutenção consome tanto tempo?

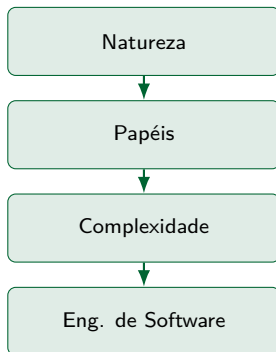
5. Como medir corretamente o progresso do projeto?



## Objetivo

Produzir software com qualidade, menor custo e tempo, maior confiabilidade e facilidade de manutenção.

- Software é produto e veículo.
- Transforma dados em decisão.
- Está em todas as áreas.
- Complexidade crescente.
- Desafios de prazo, custo e qualidade.
- A ES organiza o desenvolvimento.

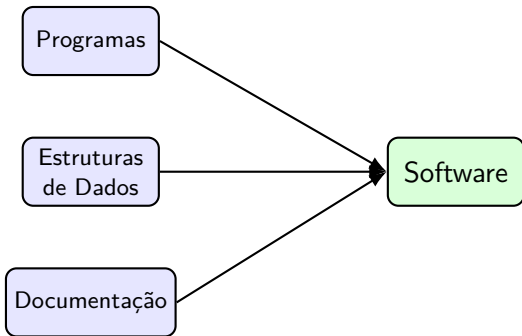


“Software é o elemento que dá inteligência ao hardware.”

## Definição

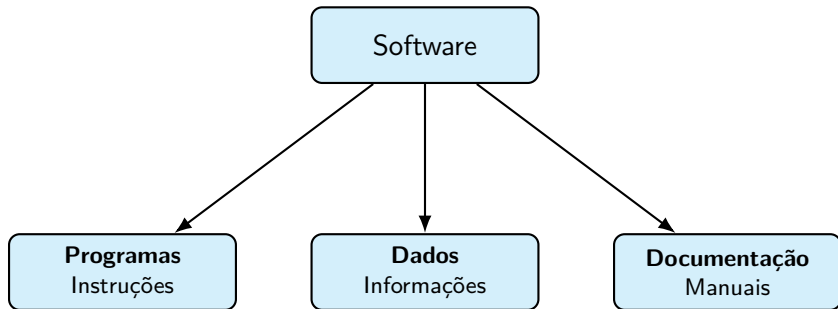
Software é composto por:

- Programas;
- Estruturas de dados;
- Documentação.



## Importante

Software não é apenas código. Dados e documentação também fazem parte do produto.

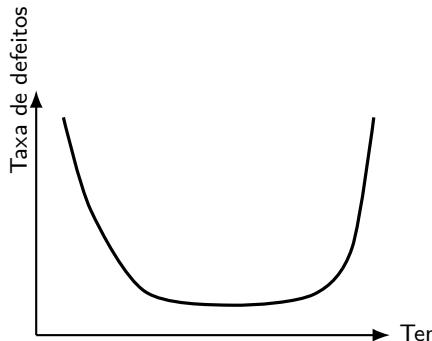


## Exemplo

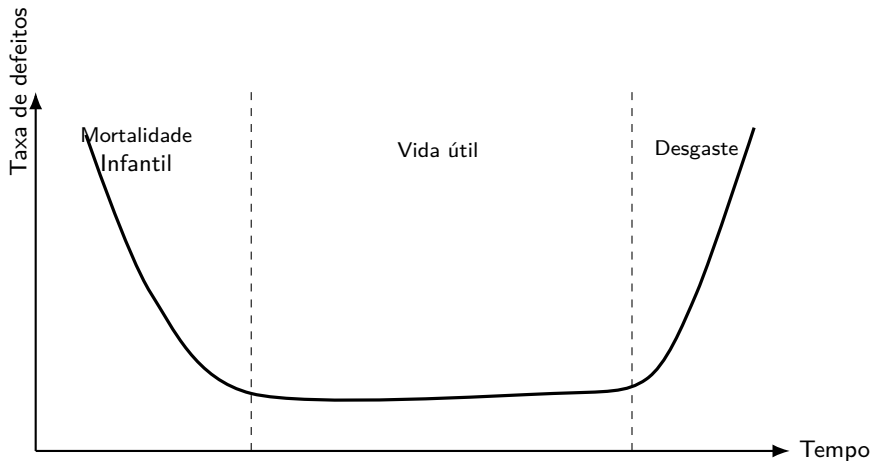
Um sistema bancário possui código-fonte, banco de dados dos clientes, documentação técnica e manual do usuário.

# A Curva da Banheira (Hardware)

- O hardware é um componente físico.
- Sua taxa de defeitos varia durante sua vida útil.
- Esse comportamento é conhecido como **Curva da Banheira**.
- A curva apresenta três fases bem definidas.



# As Três Fases da Vida do Hardware



## Interpretação

- 1 Alta taxa inicial devido a defeitos de fabricação ou projeto.
- 2 Longo período com poucas falhas.

## Hardware

- Componente físico.
- Sofre desgaste.
- Poeira.
- Vibração.
- Calor.
- Corrosão.
- Necessita substituição de peças.

## Software

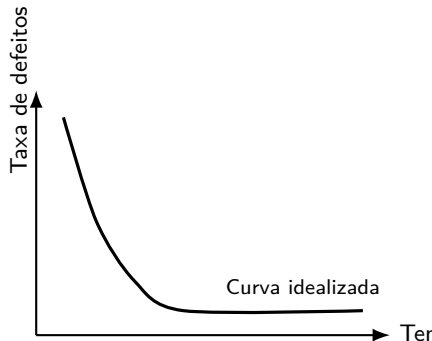
- Produto lógico.
- Não sofre desgaste físico.
- Não envelhece.
- Defeitos são erros de projeto.
- Não existem peças de reposição.

## Conclusão

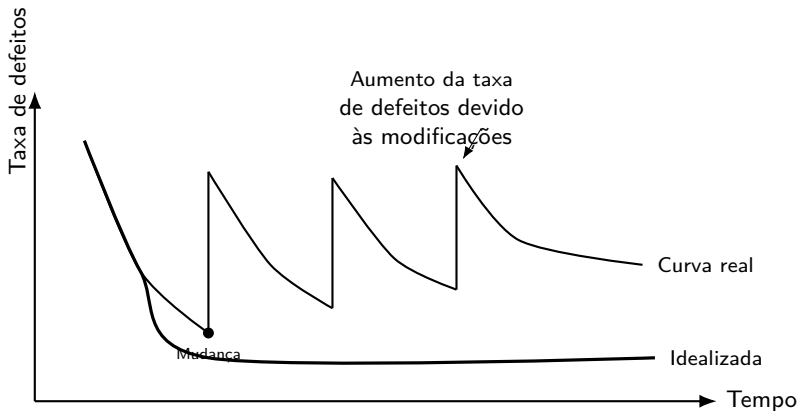
Enquanto o hardware envelhece fisicamente, o software somente muda quando é corrigido ou modificado.

# Curva Idealizada de Defeitos do Software

- Muitos defeitos são encontrados logo após o desenvolvimento.
- As correções reduzem rapidamente a taxa de falhas.
- Após as correções, espera-se uma taxa praticamente constante.
- Diferentemente do hardware, não existe desgaste físico.



# Curva Real de Defeitos do Software



## Conclusão

Cada manutenção ou nova funcionalidade pode introduzir novos erros. Assim, o software **não se desgasta**, mas pode **deteriorar devido às modificações realizadas ao longo de sua vida**.

Hardware



Desgasta-se  
fisicamente

Software



Deteriora  
pelas modificações

## Mensagem Principal

- Hardware falha devido ao envelhecimento dos componentes.
- Software não sofre desgaste físico.
- A manutenção e a evolução do software podem introduzir novos defeitos.
- Quanto maior o número de modificações, maior a necessidade de testes e controle de qualidade.

## 1.1.2 Domínios de Aplicação de Software

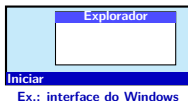
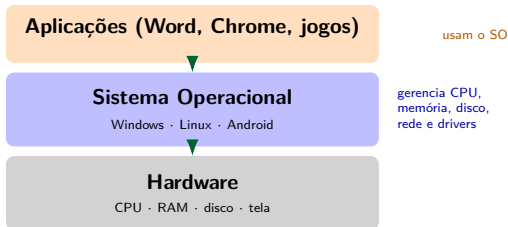
Como classificar? Observe **onde** o software roda e **para quem** ele trabalha.

| Domínio    | Onde roda?                    | Para quem?          | Exemplo famoso     |
|------------|-------------------------------|---------------------|--------------------|
| Sistema    | PC / celular<br>(camada base) | Outros programas    | Windows, Android   |
| Aplicação  | Desktop / nuvem               | Usuário final       | Word, Photoshop    |
| Científico | Workstation / HPC             | Pesquisador         | MATLAB, Jupyter    |
| Embarcado  | Dentro do aparelho            | Dispositivo         | Carro, micro-ondas |
| Web/Mobile | Navegador / app               | Milhões de usuários | Gmail, WhatsApp    |
| IA         | Nuvem + modelo                | Humanos + sistemas  | ChatGPT, Netflix   |

# 1. Software de Sistema — a “fundação”

## Ideia-chave

Sem o SO, Word/Chrome **não sobem**. O sistema operacional fica *entre* o hardware e as aplicações.



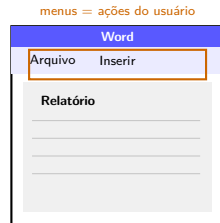
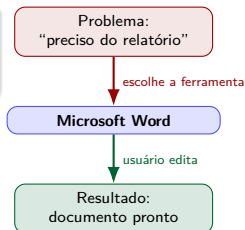
## 2. Software de Aplicação — resolve um problema

### Pergunta didática

Qual **necessidade** do usuário este programa atende?

- ❶ **Word** → escrever documentos
- ❷ **Photoshop** → editar imagens
- ❸ **App do banco** → pagar contas

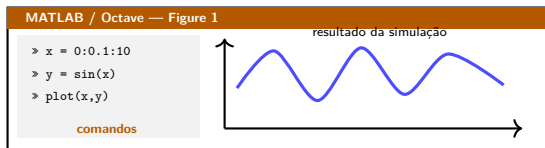
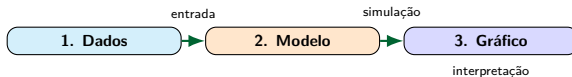
Também: ERP, SIGAA, planilhas.



### 3. Software Científico — do dado ao gráfico

#### Ideia-chave

Aqui o software é um **laboratório digital**: código → cálculo → visualização.



Exemplos: Jupyter, AutoCAD, ANSYS  
— mesma lógica: calcular e visualizar

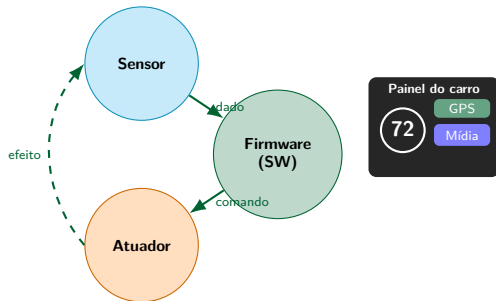
## 4. Software Embarcado — software “dentro” do aparelho

### Diferença

Você **não instala** um app: o programa já veio soldado/gravado no dispositivo.

- 1 Sensor lê o mundo
- 2 Firmware decide
- 3 Atuador age

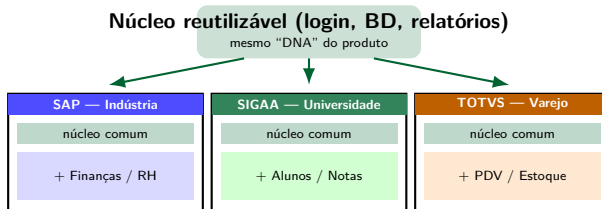
Exemplos: carro, micro-ondas, drone, equipamento médico.



## 5. Linha de Produtos — mesmo DNA, faces diferentes

### Ideia-chave

Um **núcleo comum** (reuso) + módulos **específicos** por cliente/mercado.

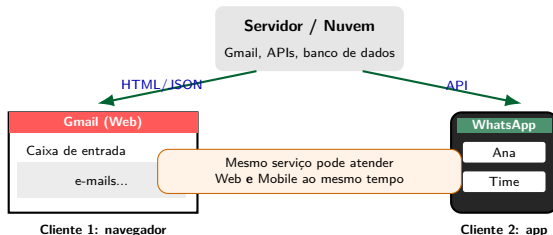


**Legenda:** faixa verde = reuso    faixa colorida = customização

## 6. Web e Mobile — um servidor, muitos clientes

### Ideia-chave

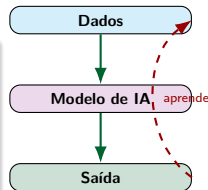
A lógica mora no **servidor**; o celular/navegador só **exibe e envia** pedidos (HTTP/API).



## 7. Inteligência Artificial — aprende com dados

### Ciclo (didático)

- 1 Coleta **dados**
- 2 Treina o **modelo**
- 3 Gera **resposta** / recomendação
- 4 Feedback melhora o modelo



ChatGPT = conversa    Netflix = recomenda filmes

### Definição

Software legado é um sistema desenvolvido há muitos anos que continua em operação e recebendo manutenção.

- Sistemas bancários
- Sistemas governamentais
- Sistemas industriais
- Sistemas hospitalares
- Sistemas acadêmicos

## Ideia-chave

Legado **funciona**, mas fica cada vez mais **caro e arriscado** de mexer.

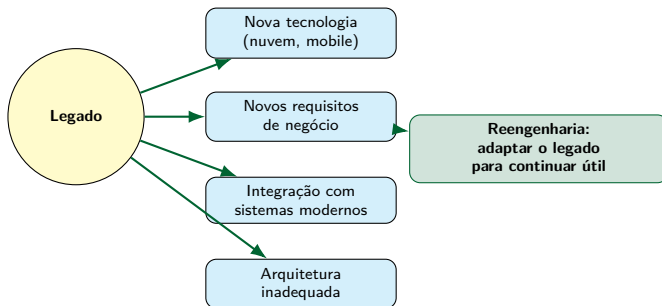


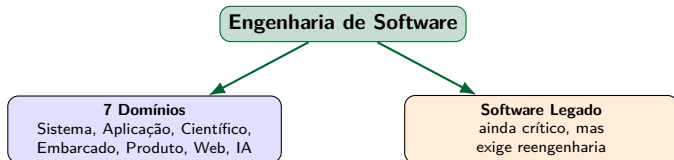
Cada mudança exige mais esforço → mais bugs → mais custo

# Quando Modernizar um Software Legado?

## Sinais de alerta

Modernize quando o sistema **impede** o negócio de avançar.





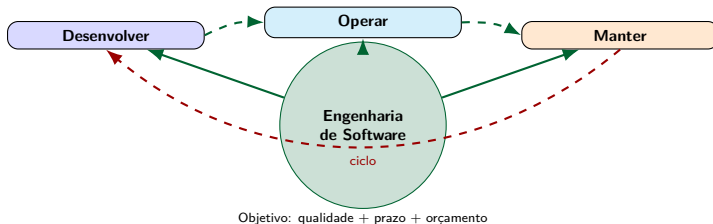
Lembrete: cada domínio tem requisitos próprios; legado precisa evoluir para não virar gargalo

- A ES atua em diversos domínios, cada um com exigências específicas.
- Sistemas legados ainda sustentam muitas organizações.
- Reengenharia prolonga a vida útil com segurança e compatibilidade.

## 1.2 Definição da Engenharia de Software

### Definição do IEEE

Abordagem **sistemática, disciplinada e quantificável** ao **desenvolvimento**, à **operação** e à **manutenção** de software.



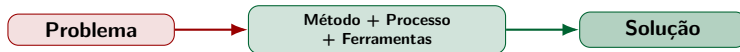
# Por que Engenharia de Software?

## Sem ES

- Código caótico
- Retrabalho
- Muitos erros
- Atrasos e custo alto

## Com ES

- Planejamento
- Padronização
- Controle
- Manutenção viável



A ES é a "ponte" entre caos e produto confiável

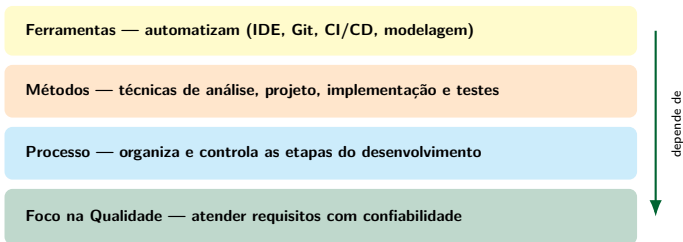


**Figura 1.3**

Camadas da engenharia de *software*.

A ES é estruturada em quatro camadas; a **qualidade** é a base de toda a disciplina.

# O Papel de Cada Camada

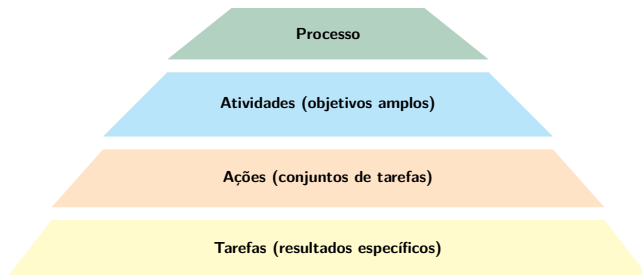


## Mensagem

Qualidade nasce da integração processo + métodos + ferramentas — não de uma camada isolada.

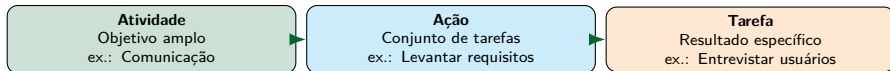
### Definição

Conjunto organizado de **atividades**, **ações** e **tarefas** para desenvolver um produto de software.



Do geral ao concreto: o processo se detalha até o trabalho diário

# Atividade, Ação e Tarefa



Analogia: **Atividade** = disciplina da faculdade; **Ação** = unidade; **Tarefa** = exercício da aula

Atividade

Comunicação com o cliente

Ação

Levantamento de requisitos

Tarefa

Entrevistar usuários

# As Cinco Atividades Metodológicas

Todo processo sério passa por estas cinco “estações” — a ordem importa.



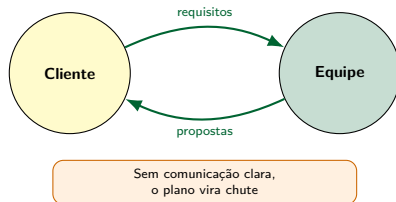
Essas atividades aparecem em cascata, iterativo e ágil — com ritmos diferentes

## Comunicação

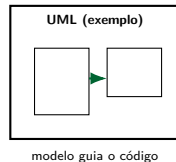
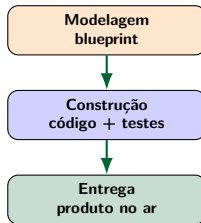
Entender o problema, conversar com o cliente, levantar requisitos e definir objetivos.

## Planejamento

Cronograma, custos, riscos e recursos.



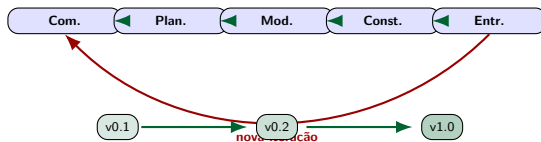
- **Modelagem:** UML, diagramas, arquitetura
- **Construção:** programação e testes
- **Entrega:** implantação e feedback



## Resultado

Cada iteração entrega um software mais completo e de maior qualidade.

Em projetos modernos, as cinco atividades **se repetem** — não acontecem uma vez só.



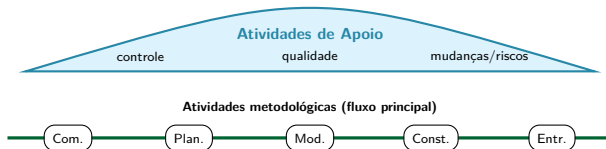
## Benefícios

Entregas frequentes · correções rápidas · feedback contínuo · menos risco

## 1.3.2 Atividades de Apoio

### Definição

**Umbrella activities:** acompanham o projeto **do início ao fim** (não são uma fase isolada).



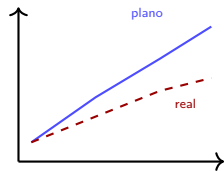
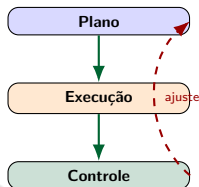
Enquanto o time desenvolve, o "guarda-chuva" protege prazo, qualidade e rastreabilidade

# Controle e Acompanhamento do Projeto

- Monitorar cronograma
- Comparar plano × execução
- Detectar atrasos cedo
- Apoiar decisões

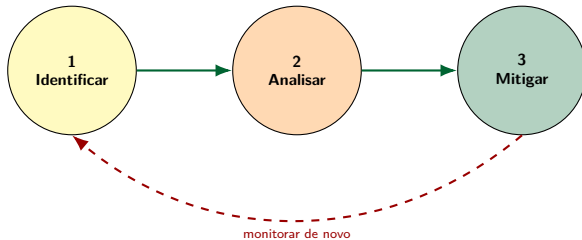
## Resultado

Corrigir desvios **antes** de virarem crise.



## Objetivo

Identificar, analisar e minimizar ameaças ao produto/projeto.



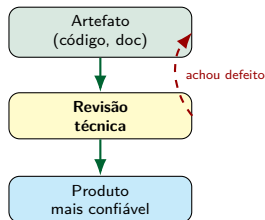
Exemplos: perda de pessoas, mudança de requisitos, atraso, falha tecnológica.

## Garantia da Qualidade

normas, padrões, auditorias, melhoria contínua

## Revisões Técnicas

requisitos, projeto, código, inspeções



## Medição

produtividade, defeitos, esforço, custo, qualidade

## Configuração

versões, histórico, mudanças, rastreabilidade



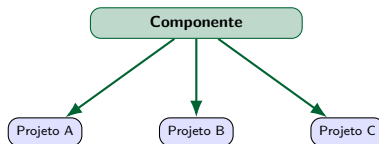
Medir para decidir; versionar para não perder o controle

## Reutilização

bibliotecas, componentes, APIs, frameworks

## Artefatos

docs, UML, testes, manuais



escrever uma vez, usar várias

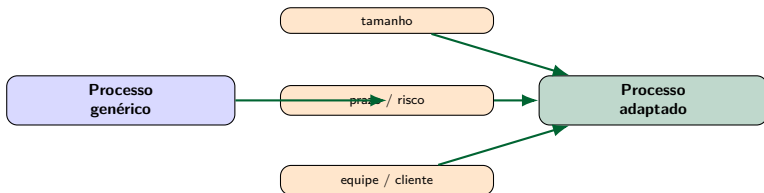
## Mensagem

Apoio ao longo do ciclo de vida  $\Rightarrow$  mais qualidade, menos risco, manutenção mais fácil.

## 1.3.3 Adaptação do Processo

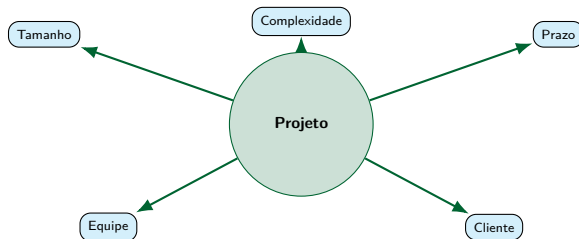
### Ideia central

Não copie o processo “de livro” no piloto automático — **customize** ao projeto, à equipe e à organização.



# Por que Adaptar o Processo?

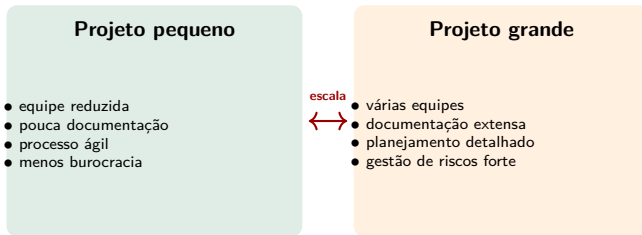
Um único processo **não** serve igualmente a todos os projetos.



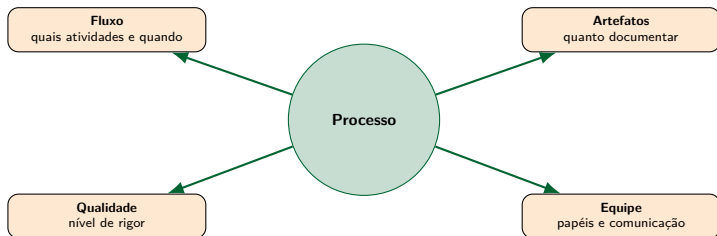
# Fatores que Influenciam a Adaptação

| Fator               | Impacto no processo                |
|---------------------|------------------------------------|
| Tamanho             | Mais documentação e controle       |
| Complexidade        | Mais análise e projeto             |
| Equipe / Cliente    | Comunicação e participação         |
| Prazo / Organização | Nº de iterações; normas da empresa |

Quanto maior a complexidade, maior tende a ser o **rigor** do processo.

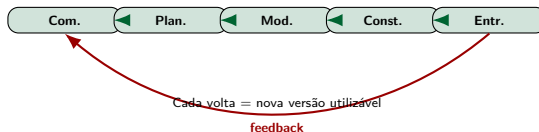


Você pode “afinar” o processo em quatro eixos:



## Caso ágil

Ciclos curtos + feedback do cliente a cada entrega.



- Não existe processo único para todos
- Flexibilidade reduz desperdício
- Rigor depende do contexto
- Meta constante: software de qualidade

**Processos eficazes se adaptam ao problema,  
não o contrário.**

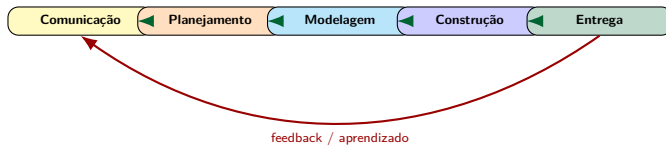
## 1.4 A Prática da Engenharia de Software

### O que é?

Aplicar **processos** + **métodos** + **ferramentas** para produzir software de forma organizada, previsível e com qualidade.



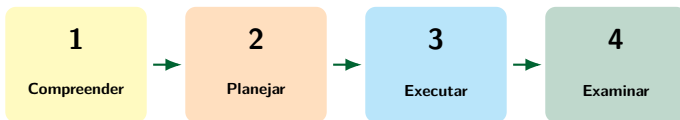
+ comunicação, planejamento, modelagem, construção, entrega e apoio



Cada entrega gera aprendizado para a próxima iteração.

## 1.4.1 A Essência da Prática

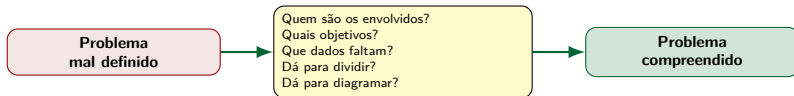
George Polya: estratégia geral de resolução de problemas — aplicável ao software.



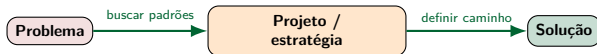
# 1º Passo: Compreender o Problema

## Objetivo

Entender certo **antes** de codar — evita retrabalho.



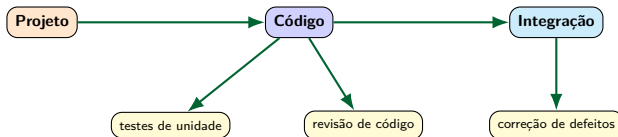
## 2º Passo: Planejar a Solução



Reuso, arquitetura adequada, divisão em subproblemas

### 3º Passo: Executar o Plano

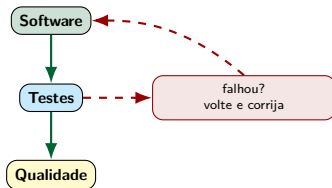
Aqui a solução vira software de verdade.

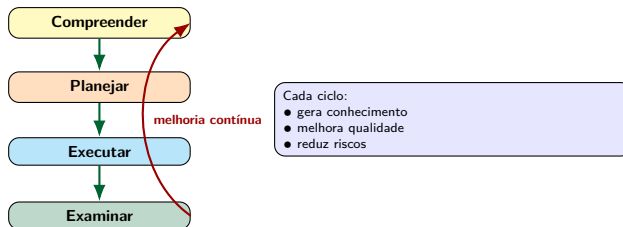


## 4º Passo: Examinar os Resultados

- Testes
- Validação
- Verificação
- Correção de erros

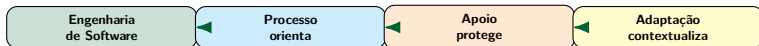
O software atende aos requisitos dos usuários?





## Mensagem

A ES é resolução de problemas **iterativa**, não uma receita linear única.



Prática: compreender → planejar → executar → examinar (e repetir)

- ES é disciplina organizada; o processo guia o desenvolvimento.
- Atividades de apoio sustentam qualidade e controle.
- O processo se adapta ao projeto.
- A prática é ciclo contínuo de resolução de problemas.